Chapter 8 Software Design Quality

Software Quality

- Software architecture is about **planning** the parts of a system that are hard to change once it's built. 软件架构是关于规划系统中一旦构建就难以更改的部分
- It includes both the steps (process) and the final structure (design) needed to build software that works correctly 构建正确运行 (meets functional needs满足功能需求) and works well (meets quality or non-functional needs).良好运行（满足质量或非功能需求）
- Quality refers to **how well a service works or how good a product is**. 指服务的运行情况或产品的品质

But quality is different for different people:

- *Customer: wants it cost-effective and works well.* 经济高效且运行良好
- *User: wants it easy and useful.* 简单易用
- *Developer: wants it easy to build and fix.* 易于构建&修复
- *Development Manager: wants it to make profit.* 能够盈利

Example: A developer might say a website is high quality because it uses neat code. But a user might disagree if the site looks ugly or doesn't work on their phone.

Evaluating Software Quality

Steps to evaluate quality:

1. **Set quality goals** – What are we trying to achieve? (e.g. easy to fix bugs)
   设定质量目标——我们想要实现什么目标？（例如，易于修复错误）
2. **Pick measurable traits** – Like how complex the code is.
   选择可衡量的特征——例如代码的复杂度。
3. **Measure those traits** – Count things like how many "if" or "while" statements are used. 衡量这些特征——计算使用了多少"if"或"while"语句。

Exp: If your goal is easy maintenance: (More complexity = harder to maintain)
You measure complexity by checking how many control paths (like "if-else") are in the code.

Software Quality Attributes

- are non-functional requirements that greatly affect how good a software system is
非功能性需求，对软件系统的优劣影响巨大
- is a feature of the system that can be measured or tested to show how well the software meets the needs and expectations of its users or stakeholders 是系统的一个特性，可以通过测量或测试来显示软件满足用户或利益相关者需求&期望的程度

Two categories:

- **Runtime attributes** (e.g. performance, availability, usability)
- **Development-time attributes** (e.g. testability, modifiability)

FIVE KEY QUALITY ATTRIBUTES

1. **RELIABILITY** 可靠性 (Does it work consistently and handle problems?)

- The designer must be able to predict how the system will work in different situations. 设计人员必须能够预测系统在不同情况下的工作方式

- **Completeness** 完整性 – Does the system perform all the tasks it is supposed to? 系统是否能够执行所有预期任务
  For example, can it handle all types of input correctly?

- **Consistency** 一致性 – Does the system always act the same way in similar situations? 系统在类似情况下是否始终以相同的方式运行
  For example, does it give the same result every time for the same input?

- **Robustness** 稳健性 – Can the system still work properly even in unusual or unexpected situations? 即使在异常或意外情况下，系统是否仍能正常工作
  For example, what happens if a required resource fails?

2. **MAINTAINABILITY**

- is **how easily** the software can be **changed** after it has been deployed.修改的难易程度
- Software may need to be updated or modified for several reasons:修改的难易程度
  - **Fixing errors** – Some bugs or issues might not be found during testing and only appear after the software is in use. 某些错误或问题可能在测试期间未发现，只有在软件投入使用后才会出现。
  - **Improving performance** – Performance problems may only show up once the software is running in a real-world environment. 性能问题可能只有在软件在实际环境中运行时才会出现
  - **Changing requirements** – The user's needs or system requirements might change, and the software must be updated to meet those new needs. 用户需求或系统要求可能会发生变化，软件必须更新以满足这些新需求

3. **EFFICIENCY** 效率

- refers to **how well** the software **uses system resources** like memory, processing power, and network bandwidth. 是指软件如何高效地利用内存、处理能力&网络带宽等系统资源
- In most cases, efficiency is considered **less important than** reliability, but it still matters for overall system performance. 效率的重要性不如可靠性

4. USABILITY 可用性

- is about how easy it is for a user to complete a task using the software. 完成任务的难易程度
- It includes several important aspects:
  - **Learning system features** 学习系统功能 – How quickly and easily users can understand how to use the software. 理解软件使用方法的 速度&难易程度
  - **Using the system efficiently** 高效使用系统 – How smoothly users can perform tasks once they know how to use the system. 掌握系统使用方法后，执行任务的流畅程度
  - **Minimizing the impact of errors** 最大限度地减少错误的影响 – How well the system helps users avoid mistakes or recover from them.系统如何有效地帮助用户 避免错误或从错误中恢复
  - **Adapting the system to user needs** 根据用户需求调整系统 – How flexible the software is in adjusting to different user preferences or workflows. 软件在适应不同用户偏好或工作流程方面的灵活性
  - **Increasing confidence and satisfaction** 提升用户信心&满意度 – How comfortable and satisfied users feel while using the software. 用户在使用软件时,感到舒适&满意的程度

5. REUSABILITY 可重用性

- is about how easily parts of the software **can be used again** in different applications or systems. 是指软件的 某些部分在不同应用程序或系统中 重复使用的难易程度
- It focuses on how simple it is to:
  - **Adapt software components for other purposes** 将软件组件用于其他用途 – Use existing code or modules in new projects with little or no change.
    在新项目中几乎不做任何改动或使用现有代码或模块。

OTHER QUALITY ATTRIBUTES

1. **AVAILABILITY** 可用性

- Is the system ready to use when needed? 系统是否在需要时即可使用
- is about how often the system is working and ready for use, especially after something goes wrong. 是指系统正常运行并随时可用的频率，尤其是在出现故障后
- It includes:
    - **Handling system failures** 处理系统故障 – It looks at how failures affect users or other connected systems. 关注故障如何影响用户或其他连接系统
    -
    - **Recovering from faults** 故障恢复 – Builds on reliability by focusing on how the system can recover and keep working, even after a problem在可靠性的基础上，关注系统如何在出现问题后恢复并继续工作 (e.g., a fault-tolerant system 容错系统).
    - **Minimizing downtime** 最小化停机时间 – Reduces the time the system is unavailable by quickly detecting and fixing issues. 通过快速检测&修复问题来减少系统不可用的时间
- A failure happens when the system doesn't behave as it was designed to. Availability ensures the system stays usable as much as possible, even when faults occur. 系统即使在发生故障时也能尽可能保持可用

Availability Tactics 可用性策略

- are strategies used to keep the system running and reduce downtime when problems happen. 在发生问题时，用于保持系统正常运行并减少停机时间的策略
- They include:

- **Fault Detection** 故障检测 – Identifying if parts of the system are still working properly. 识别系统各部分是否仍在正常运行
  Example: Using tools like *ping* or *self-tests* to check if a component is alive.

- **Fault Recovery** 故障恢复 – Taking action to fix or work around a problem once it's found. 发现问题后立即采取措施修复或规避问题
  Example: Retrying a failed operation or using *exception handling*, or relying on backup data or systems.

- **Fault Prevention** 故障预防 – Reducing the chance of faults or limiting their impact. 降低故障发生的可能性或限制其影响
  Example: Temporarily removing a faulty component from use to prevent it from affecting the rest of the system.

2. Interoperability 互操作性

- is the ability of two or more systems to exchange meaningful information and use it effectively. 两个或多个系统交换有意义的信息并有效使用这些信息的能力
- For systems to be interoperable, they must:
    - **Exchange data** – Share information with each other.
      交换数据——相互共享信息
    - **Understand the data** – Present the shared data in a way that users or systems can understand and use.
      理解数据——以用户或系统能够理解和使用的方式呈现共享数据
- Achieving interoperability requires the systems to find each other and manage how they connect and communicate. 需要系统能够相互查找并管理它们的连接和通信方式

<u>Interoperability Tactics</u>

To support interoperability, the following tactics are used:

- **Locate Interfaces** 定位接口 – Helps systems find each other during runtime.
  帮助系统在运行时相互查找
  Example: Using a *service discovery* mechanism.

- **Manage Interfaces** 管理接口 – Ensures that systems can properly handle how they connect and share data. 确保系统能够正确处理连接和数据共享

- **Orchestrate** 编排 – Uses a control mechanism to manage and coordinate communication between systems. 使用控制机制管理和协调系统间的通信

- **Tailor Interface** 定制接口 – Modifies the interface by adding or removing features. 通过添加或删除功能来修改接口

3. Modifiability 可修改性

- is about how easily software can be changed during or after its initial development. 初始开发期间或之后修改的难易程度
- It focuses on making changes with minimal cost in terms of time, money, or effort. 最小的时间、金钱或精力成本进行修改
- Reasons for change include:
    - Adopting **new technologies**, platforms, protocols, or standards 采用新技术、平台、协议或标准
    - **Adding** new features, or **changing/removing** old ones 添加新功能，或更改/删除旧功能

- **Fixing bugs**, improving **security**, or boosting **performance** 修复错误、提高安全性或提升性能
- Making systems work **better together** 使系统更好地协同工作
- Improving the **user experience** 提升用户体验

Modifiability Tactics

a. Reduce Module Size 缩减模块大小

- **Split large modules** into smaller, more focused ones 将大型模块拆分为更小、更专注的
  → Makes future changes easier and cheaper. 使未来的修改更容易、更经济

b. Increase Cohesion 增强内聚性

- Move related responsibilities into the same module. 将相关职责移至同一模块

- If tasks A and B in a module serve different purposes, they should be separated.
  → This might involve **creating a new module** or **moving responsibilities** to existing ones. 这可能涉及 创建新模块或将职责移至现有模块

c. Reduce Coupling 降低耦合

- **Encapsulate** details to hide internal logic and reduce dependency on other modules.
  封装细节以隐藏内部逻辑并减少对其他模块的依赖
- Use **intermediaries** to break direct dependencies between modules.
  使用中介来打破模块之间的直接依赖关系
- Apply **refactoring** to improve design without changing behavior.
  应用重构来改进设计，而无需改变行为

d. Defer Binding 延迟绑定

Delay the decision of how components are connected or used until later stages (e.g., runtime). 将组件的连接或使用方式的决定推迟到后续阶段（例如，运行时）

- **Runtime registration** 运行时注册
- **Plug-ins** (components loaded when needed) 插件
- **Polymorphism** (using interfaces or base classes to allow flexibility) 多态性（使用接口或基类来实现灵活性）

4. **Performance** 性能

- is the ability of a system to meet timing requirements. 系统满足时序要求的能力
- Performance is especially important in **real-time systems**, where timing is critical.
- It is usually measured by:
    - **Throughput** 吞吐量– How much work the system can do in a certain amount of time. 系统在一定时间内可以完成的工作量
    - **Response time** 响应时间 – How quickly the system reacts to a request. 系统对请求的响应速度
- **Response Time** includes:
    - **Processing Time** 处理时间 – The time the system spends actively working to respond 系统主动响应的时间. This uses CPU, memory, or other resources. 占用资源
    - **Blocked Time** – The time the system is waiting 系统等待的时间, either because:
        - A needed resource is unavailable 所需资源不可用
        - It is waiting for the result of another computation. 正在等其他计算的结果

How to Improve Performance

1. Reducing Demand 减少需求

- Lower the amount of work the system has to do. 减少系统需要完成的工作量
- May involve reducing output quality (fidelity) or rejecting some requests. 降低输出质量（保真度）或拒绝某些请求

2. Managing Resources 管理资源

- Use strategies like:
    - **Scheduling** 调度 – Prioritize tasks efficiently 高效地确定任务优先级
    - **Replication** 复制 – Run the same service on multiple systems 在多个系统上运行相同的服务
    - **Increasing resources** 增加资源 – Add more memory, processing power, etc. 添加更多内存、处理能力等

5. **Security** 安全性

- is a measure of how well a system protects its data and services from unauthorized access 衡量系统在允许授权用户和系统访问的同时, while still allowing access to authorized users and systems.如何有效保护其数据和服务免受未经授权的访问
- It reflects how well the system is designed to resist attacks and prevent misuse.它反映了系统抵御攻击和防止滥用的设计能力
- Security ensures that the system remains trustworthy, even in the face of threats or attacks.确保系统即使在面临威胁或攻击的情况下也能保持可信

Three Key Characteristics of Security:

- **Confidentiality** 机密性 – Ensures that data and services are only accessible to authorized users. 确保只有授权用户才能访问数据和服务
- **Integrity** 完整性 – Ensures that data and services cannot be changed or tampered with by unauthorized users. 确保数据和服务不会被未经授权的用户更改或篡改
- **Availability** 可用性 – Ensures that authorized users can access the system and its services when needed. 确保授权用户可以在需要时访问系统及其服务

Potential Security Threats:

- ❖ **System Penetration** 系统渗透 – An unauthorized person tries to access the system and perform actions they are not allowed to. 未经授权的人员试图访问系统并执行其无权执行的操作
- ❖ **Authorization Violation** 授权违规 – An authorized user misuses their access to the system. 授权用户滥用其系统访问权限
- ❖ **Confidentiality Disclosure** 机密性泄露 – Sensitive or secret information is exposed to someone who should not have access to it. 敏感或机密信息被泄露给不应访问的
- ❖ **Denial of Service (DoS)** 拒绝服务 – Legitimate users are prevented from accessing the system, often by overloading it or causing it to crash. 合法用户被阻止访问系统，通常是通过导致系统过载或崩溃来实现的
- ❖ **Repudiation** 否认 – A user denies having done something (like a transaction), even though they did, trying to avoid responsibility. 用户否认自己执行过某项操作（例如交易），试图逃避责任

6. Testability 可测试性

- is the degree to which a software system can be tested to **ensure it works correctly**. 软件系统可测试性程度，以确保其正常运行
- **High testability** makes it easier to detect and fix bugs, verify functionality, and ensure quality. 使检测和修复错误、验证功能和确保质量变得更加容易
- It is important to create a **software test plan early** in the development process. 在开发过程的早期制定软件测试计划至关重要

- Good testability allows problems to be found earlier and fixed more easily, improving software quality and reducing development costs. 可以更早地发现问题并更轻松地解决问题，从而提高软件质量并降低开发成本

Test Planning by Development Phase:

Requirements Phase

→ Create **functional test cases** (also called black-box tests).

→ Based on the **use case model** to check if the system does what the user expects. 基于用例模型检查系统是否符合用户的预期

Architectural Design Phase

→ Create **integration test cases**.

→ Focus on testing how **components interact** and whether the interfaces between them work properly. 重点测试组件之间的交互方式以及接口是否正常工作

Detailed Design & Coding Phase

→ Create **white-box test cases**.

→ Test the **internal logic and algorithms** of each component to ensure they function correctly. 测试每个组件的内部逻辑和算法，确保其正常运行

7. Scalability 可扩展性

- The ability of the system to **grow or expand** after it is deployed. 增长或扩展的能力。
- Can handle more users, data, or workload without major changes. 无需进行重大更改即可处理更多用户、数据或工作负载

8. Portability 可移植性

- How easily the software can be moved from one platform (e.g., operating system or hardware) to another. 软件在不同平台（例如操作系统或硬件）之间迁移的难易程度
- Important for systems that may need to run in different environments. 对于可能需要在不同环境中运行的系统而言至关重要

9. Traceability 可追溯性

- The ability to track each product or output (like code or test cases) back to earlier stages of development, such as requirements or design. 将每个产品或输出（例如代码或测试用例）追溯到开发早期阶段（例如需求或设计 ）的能力
- Helps ensure that all requirements are fulfilled and changes are well-documented.有助于确保所有需求都得到满足，并且所有变更都得到妥善记录

10. Flexibility 灵活性

- The system's ability to be configured or adjusted to suit different conditions or needs. 系统根据不同条件或需求进行配置或调整的能力
- Useful for handling a wide range of use cases without needing major changes to the code. 有助于处理各种用例，而无需对代码进行重大更

11. Manageability 可管理性

- How well the system's design supports planning, tracking, and estimating the effort needed for development and maintenance. 系统设计对开发和维护所需工作的规划、跟踪和估算的支持程度
- Helps teams understand and manage the complexity of implementation 帮助团队理解和管理实施的复杂性

12. Buildability 可构建性

- How easily the design can be turned into actual working software. 设计转化为实际工作软件的难易程度
- A buildable design is clear, well-structured, and easy to implement for developers. 可构建的设计清晰、结构良好，且易于开发人员实施

Measurable attributes of software

1. Simplicity 简单性

- "A solution should be as simple as possible, but no simpler." 尽可能简单，但不能更简单
- Simplicity in software means the **design should do what it's supposed to** — nothing more, nothing less. 设计应该完成其预期的功能——不多不少
- It should not include unnecessary complexity or extra features. 不应包含不必要的复杂性或额外的功能
- Why Simplicity Matters: →Makes software **easier to maintain** 使软件更易于维护
  → Increases **reusability** of components 提高组件的可重用性
  → Improves **testability** 提高可测试性
- **How to Measure Simplicity (by identifying complexity):**

- **Control flow complexity** 控制流复杂度 – Number of possible paths through the code (more paths = more complex). 代码中可能路径的数量（路径越多 = 越复杂）

- **Information flow complexity** 信息流复杂度 – Number of data items shared between parts of the program. 程序各部分之间共享的数据项数量

- **Namespace complexity** 命名空间复杂度 – Number of unique names (identifiers and operators) used in the code.代码中使用的唯一名称（标识符和操作符）的数量

2. Modularity 模块化

- Modularity means breaking down a system into smaller, manageable parts (modules), each with a clear responsibility. 分解成更小、更易于管理的部分（模块），每个部分都具有明确的职责
- Key Principles:
  - Each module handles **one concern**. 每个模块处理一个问题
  - Modules communicate through **well-defined interfaces**. 模块通过定义明确的接口进行通信
  - A good design has **low coupling** (few dependencies) and **high cohesion** (module parts work closely together). 具有低耦合（依赖关系少）和高内聚（模块各部分紧密协作）的特点
  -
- Benefits of Modularity:
  - Modules are easier to **understand, reuse, replace**, and **test**.模块更易于理解、复用、替换和测试
  - Simplifies maintenance and future updates. 简化维护和未来更新
- How to Measure Modularity:

- **Cohesion** – How strongly the elements within a module are related. (Higher is better) 模块内元素之间的关联强度

- **Coupling** – How dependent modules are on each other. (Lower is better) 模块之间的依赖程度

3. Information Hiding

- means keeping internal details of a module (like data structures and algorithms) hidden from other modules. 是指将模块的内部细节（例如数据结构和算法）隐藏起来，不让其他模块看到
- What it Involves:
  - Hiding *how* a module does something and only showing *what* it does (interface). 隐藏模块的功能，仅显示其功能（接口）

  - Using **access methods** (like getters/setters) instead of exposing data directly. 使用访问方法,而不是直接暴露数据

- Benefits:
  - Prevents other modules from relying on internal details. 防止依赖内部细节

  - Protects data from being accidentally changed (reduces bugs). 保护数据免遭意外更改

  - Makes the system more **robust and consistent**. 使系统更加健壮和一致

- How It Helps:
  - Reduces **dependency** between modules. 减少模块之间的依赖关系

  - Makes the software **more flexible and maintainable**. 使软件更加灵活和易于维护